LAB: GPIO 7-segment(eval board)

LAB: GPIO 7-segment(eval board)

LAB: GPIO 7-segment

Date: 2026-09-02

Author/Partner: Juyoung Lee

Demo Video: Youtube link

PDF version:

"LAB_GPIO_7segment_22200576_JuyoungLee.pdf" could not be found.

Introduction

In this lab, you are required to create a simple program to control a 7-segment display to show a decimal number (0~9) that increases by pressing a push-button.

Exercise

Port/Pin	Description	Register setting
Port B Pin 5	Clear Pin5 mode	GPIOB→MODER &=~(3<<(5*2))
Port B Pin 5	Set Pin5 mode = Output	GPIOB→MODER |=(1<<(5*2))
Port B Pin 6	Clear Pin6 mode	GPIOB→MODER &=~(3<<(6*2))
Port B Pin 6	Set Pin6 mode = Output	GPIOB→MODER |=(1<<(6*2))
Port B Pin Y	Clear PinY mode	GPIOB→MODER &=~(3<<(Y*2))
Port B Pin Y	Set PinY mode = Output	GPIOB→MODER |=(1<<(Y*2))
Port B Pin 5~9	Clear Pin5~9 mode	GPIOB→MODER &=~(1023<<(5*2))
	Set Pin5~9 mode = Output	GPIOB→MODER |=(341<<(5*2))
Port X Pin Y	Clear Pin Y mode	GPIOX→MODER &=~(3<<(Y*2))
	Set Pin Y mode = Output	GPIOX→MODER |=(1<<(Y*2))
Port B Pin5	Set Pin5 otype=push-pull	GPIOB→OTYPER &=~(1<<(5))
Port B PinY	Set PinY otype=push-pull	GPIOB→OTYPER&=~(1<<(Y))
Port B Pin5	Set Pin5 ospeed=Fast	GPIOB→OSPEEDR |=(2<<(5*2))
Port B PinY	Set PinY ospeed=Fast	GPIOB→ OSPEEDR |=(2<<(Y*2))
Port B Pin 5	Set Pin5 PUPD=no pullup/down	GPIOB→OTYPER &=~(3<<(5*2))
Port B Pin Y	Set PinY PUPD=no pullup/down	GPIOB→ OTYPER &=~(3<<(Y*2))

Problem 1. Library for 7-segment display on JKIT Board

```
// Display a number 0 - 9 only
void FND_display_init(PinName_t *pinFND);
void FND_display(uint8_t  num, PinName_t *pinFND)

// Select display: FND0 to FND3
void FND_select_init(PinName_t *selectFND);
void FND_select(uint8_t select, PinName_t *selectFND);
```

Function 1. FUD_display_init()

1. **brief**
    1. Initializes GPIO pins for 7-segment display (FND) control.
2. **details**
    1. Configures 8 segment pins as digital output, push-pull, no pull-up/pull-down, and medium speed. Stores the pin mapping in the internal global array `_fndPins`.
3. **param(input)**
    1. pinFND: Pointer to an array of 8 PinName_t elements (Segment A~G, DP).
4. **return**
    1. None
5. **note Assumes**
    1. `_fndPins` array is pre-allocated with a minimum size of 8.

```
// Initialize DOUT pins for 7 segment leds
void FND_display_init(PinName_t *pinFND){
    for (int i=0; i<8; i++){
        _fndPins[i] = pinFND[i];
        GPIO_init        (_fndPins[i], OUTPUT);
        GPIO_pupd       (_fndPins[i], NON);
       GPIO_otype      (_fndPins[i], PUSH_PULL);
       GPIO_ospeed     (_fndPins[i], MEDIUM);
    }
}
```

Function 2. FND_select_init()

1. **brief**
    1. Initialize select pins (FND0~FND3) as DOUT
2. **details**
    1. Configures 4 digit selection pins as digital output, push-pull, no pull-up/pull-down, and medium speed. Stores the pin mapping in the internal global array `_fndSel`.

3. **param(input)**
    1. selectFND Pointer to an array of 4 PinName_t elements (Digit Select Pins 0~3).
4. **return**
    1. None
5. **note Assumes**
    1. `_fndSel` global array is pre-allocated with a minimum size of 4.

```
// Initialize select pins (FND0~FND3) as DOUT
void FND_select_init(PinName_t *selectFND){
    for (int i=0; i<4; i++){
        _fndSel[i] = selectFND[i];
        GPIO_init        (_fndSel[i], OUTPUT);
        GPIO_pupd        (_fndSel[i], NON);
        GPIO_otype       (_fndSel[i], PUSH_PULL);
        GPIO_ospeed      (_fndSel[i], MEDIUM);
    }
}
```

Function 3. FND_display()

1. **brief**
    1. Displays a single digit (0–9) on the 7-segment display (FND).
2. **details**
    1. Drives the 8 segment pins (A~G, DP) by applying the pin states retrieved from the `numberFND` lookup table based on the input digit.
3. **param(input)**
    1. num Single digit value to display (valid range: 0 ~ 9).
4. **return**
    1. None
5. **note Assumes**
    1. The 7-segments on JKIT is Common Cathode
    2. `_fndPins` and `numberFND` lookup table are properly initialized. Ensure `num` does not exceed 9 to prevent out-of-bounds array access.

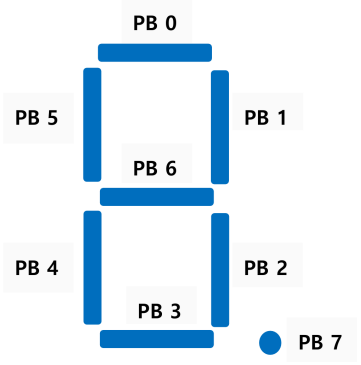
```
// Display a number 0 - 9 only
void FND_display(uint8_t num){
    for (int i=0; i<8; i++){
        GPIO_write(_fndPins[i], numberFND[num][i]);
    }
}
```

Function 4. FND_select()

1. **brief**
    1. Enables a specific digit (FND0~FND3) for 7-segment display multiplexing.
2. **details**
    1. Iterates through the 4 digit selection pins in `_fndSel`. Sets the pin corresponding to `digit` to `HIGH` (active) and drives all other digit pins `LOW` (inactive).
3. **param(input)**
    1. digit Index of the digit position to activate (valid range: 0 ~ 3).
4. **return**
    1. None
5. **note Assumes**
    1. `_fndSel` global array has been initialized using `FND_select_init()`. If `digit` is greater than 3, all digits will be set to `LOW` (disabled).

```
// Select display: FND0 to FND3
void FND_select(uint8_t digit){
    for(int i = 0; i < 4; i++){
        if(i == digit) GPIO_write(_fndSel[i], HIGH);
        else GPIO_write(_fndSel[i], LOW);
    }
}
```

Configuration



Function	Port - Pin	Configuration
Selection of 7-Segment Display	PA_10, PA_11, PC_4, PC_3 (FND_0~FND_3)	DOUT, Push-Pull,
7-Segment LEDs	PB_0, PB_1, PB_2, PB_3, PB_4, PB_5, PB_6, PB_7 ('a'~'g''dot', respectively)	DOUT, Push-Pull, Medium Speed

Example 1. Display '50' on an array of 7-segment displays.

1. `#define`
    1. BUTTON_PIN: PA_4
    2. FND Segment Pins: PB_0 – PB_7 (Segments A through G, DP)
    3. FND Select Pins: PA_10, PA_11, PC_4, PC_3 (Digits FND0 through FND3)
2. **setup()**
    1. Initializing the system clock before configuring the FND and button GPIO pins follows the proper execution order.
3. **main()**
    1. Declaring variables for the target number, individual digits (`5` and `0`), and active digit selection index here covers all required state tracking.
    2. loop()
4. **loop()**
    1. Rapidly alternating between displaying `5` on the tens digit and `0` on the ones digit relies on Persistence of Vision to create a solid `50` .

```
/*-----------------------------------------------------------------\
@ Embedded Controller by Young-Keun Kim - Handong Global University
Author           : [ Juyoung Lee ]
Created          : 09-11-2026
Modified         : 09-12-2026
Language/ver     : C in VS Code

Description      : Control 7-segments display
/-----------------------------------------------------------------*/

#include "stm32f4xx.h"
#include "ecGPIO2.h"
#include "ecRCC2.h"

#define BUTTON_PIN PA_4

PinName_t pinFND[8]     = {PB_0, PB_1, PB_2, PB_3, PB_4, PB_5, PB_6, PB_7};
PinName_t selectFND[4]  = {PA_10, PA_11, PC_4, PC_3};


void setup(void){
    // 1) Initialize System Clock
    RCC_HSI_init();

    // 2) Initialize FND pins and Others
    FND_display_init(pinFND);
    FND_select_init(selectFND);

    GPIO_init(BUTTON_PIN, INPUT);
```

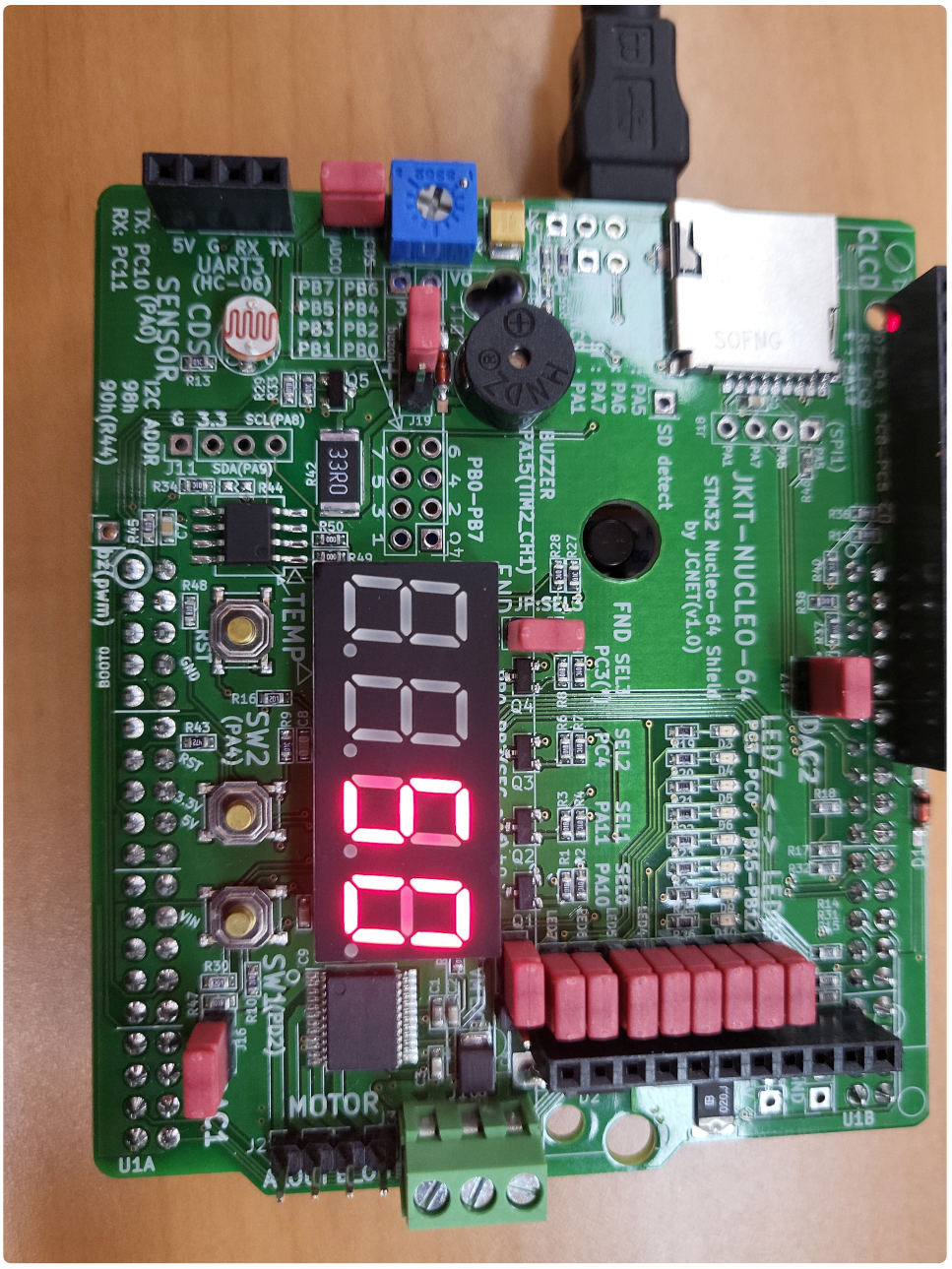
```
};

int main(void) {
    setup();
    uint8_t numDisplay1  = 0;
    uint8_t numDisplay2  = 5;
    uint8_t selectFND1  = 0;
    uint8_t selectFND2  = 1;

    while (1) {
        FND_select(selectFND1);
        FND_display(numDisplay1);
        delay_ms_HSI(1);
        FND_select(selectFND2);
        FND_display(numDisplay2);
        delay_ms_HSI(1);
    }

}
```

```
extern int numberFND[12][8];
void FND_init(PinName_t *pinFND);
void FND_select_init(PinName_t *selectFND);
void FND_display(uint8_t num);
void FND_select(uint8_t digit);
```

```
/// FND - JKIT board
static PinName_t _fndPins[8];
static PinName_t _fndSel[4];

//Each led that has to light up gets a 1, every other led gets a 0
//its in order  { (a,b,c,d,e,f,g,dp) }
int numberFND[12][8]={
                    {1,1,1,1,1,1,0,0},          //zero
                    {0,1,1,0,0,0,0,0},          //one
                    {1,1,0,1,1,0,1,0},          //two
                     {1,1,1,1,0,0,1,0},             //three
                     {0,1,1,0,0,1,1,0},             //four
                     {1,0,1,1,0,1,1,0},             //five
                     {1,0,1,1,1,1,1,0},             //six
                     {1,1,1,0,0,1,0,0},             //seven
                     {1,1,1,1,1,1,1,0},             //eight
                     {1,1,1,1,0,1,1,0},             //nine
                    {0,0,0,0,0,0,0,1},          //dot
                     {0,0,0,0,0,0,0,0}              //blank
                };
```

```
// Initialize DOUT pins for 7 segment leds
void FND_display_init(PinName_t *pinFND){
    for (int i=0; i<8; i++){
        _fndPins[i] = pinFND[i];
        GPIO_init        (_fndPins[i], OUTPUT);
        GPIO_pupd       (_fndPins[i], NON);
       GPIO_otype      (_fndPins[i], PUSH_PULL);
       GPIO_ospeed     (_fndPins[i], MEDIUM);
    }
}

// Initialize select pins (FND0~FND3) as DOUT
void FND_select_init(PinName_t *selectFND){
    for (int i=0; i<4; i++){
        _fndSel[i] = selectFND[i];
        GPIO_init        (_fndSel[i], OUTPUT);
        GPIO_pupd       (_fndSel[i], NON);
       GPIO_otype      (_fndSel[i], PUSH_PULL);
       GPIO_ospeed     (_fndSel[i], MEDIUM);
    }
}

// Display a number 0 - 9 only
void FND_display(uint8_t num){
    for (int i=0; i<8; i++){
        GPIO_write(_fndPins[i], numberFND[num][i]);
    }
}

// Select display: FND0 to FND3
void FND_select(uint8_t digit){
    for(int i = 0; i < 4; i++){
        if(i == digit) GPIO_write(_fndSel[i], HIGH);
        else GPIO_write(_fndSel[i], LOW);
    }
}
```

Result



Problem 2: Counter with Button Press

Procedure

Create a code that increases the displayed number from 0 to 19 with each button press.

- After the number '19', it should start from '0' again.
- Modify from Problem 1 source code

Configuration

Function	Port - Pin	Configuration
Button (SW2) on JKIT	PA_4	DIN, Pull-Up
7-Segment DOUT	PB_0, PB_1, PB_2, PB_3, PB_4, PB_5, PB_6, PB_7	Push-Pull, No Pull-up-Pull-down, Medium Speed

Function	Port - Pin	Configuration
	('a'~'g''dot', respectively)	
Selection of 7-Segment Display	PA_10 (FND_0)	DOUT, Push-Pull,

*** Challenge : Extend the display number from 0 to 19**

State Table

state	Next state(button = 0)	Next state(button = 1)	output
S0	S0	S1	7seg display =00
S1	S1	S2	7seg display =01
S2	S2	S3	7seg display =02
...	...	...	...
S9	S9	S10	7seg display =09
S10	S10	S11	7seg display =10
S11	S11	S12	7seg display =11
...	...	...	...
S19	S19	S0	7seg display =19

Source code (LAB_GPIO_7segment)

1. **define**
   - **BUTTON_PIN:** GPIO pin connected to the push button
   - **BUTTON_DLAY:** Button debouncing delay to prevent mechanical contact chatter [ms]
   - **SEVEN_SEG_DLAY:** Time-division multiplexing delay per digit to prevent ghosting/overlap [ms]
   - **NUM_STATE:** Total number of display states (0 to 19)
2. **Global Variables**
   - **pinFND:** Array of GPIO pins connected to each LED segment (a–g, dp) in the 7-segment display
   - **selectFND:** Array of GPIO pins used to enable/select a specific 7-segment digit
   - **numDisplay1, numDisplay2:** Numerical values to display on digit 1 (units) and digit 2 (tens)

- **selectFND1, selectFND2:** Digit selector indices corresponding to display 1 and display 2

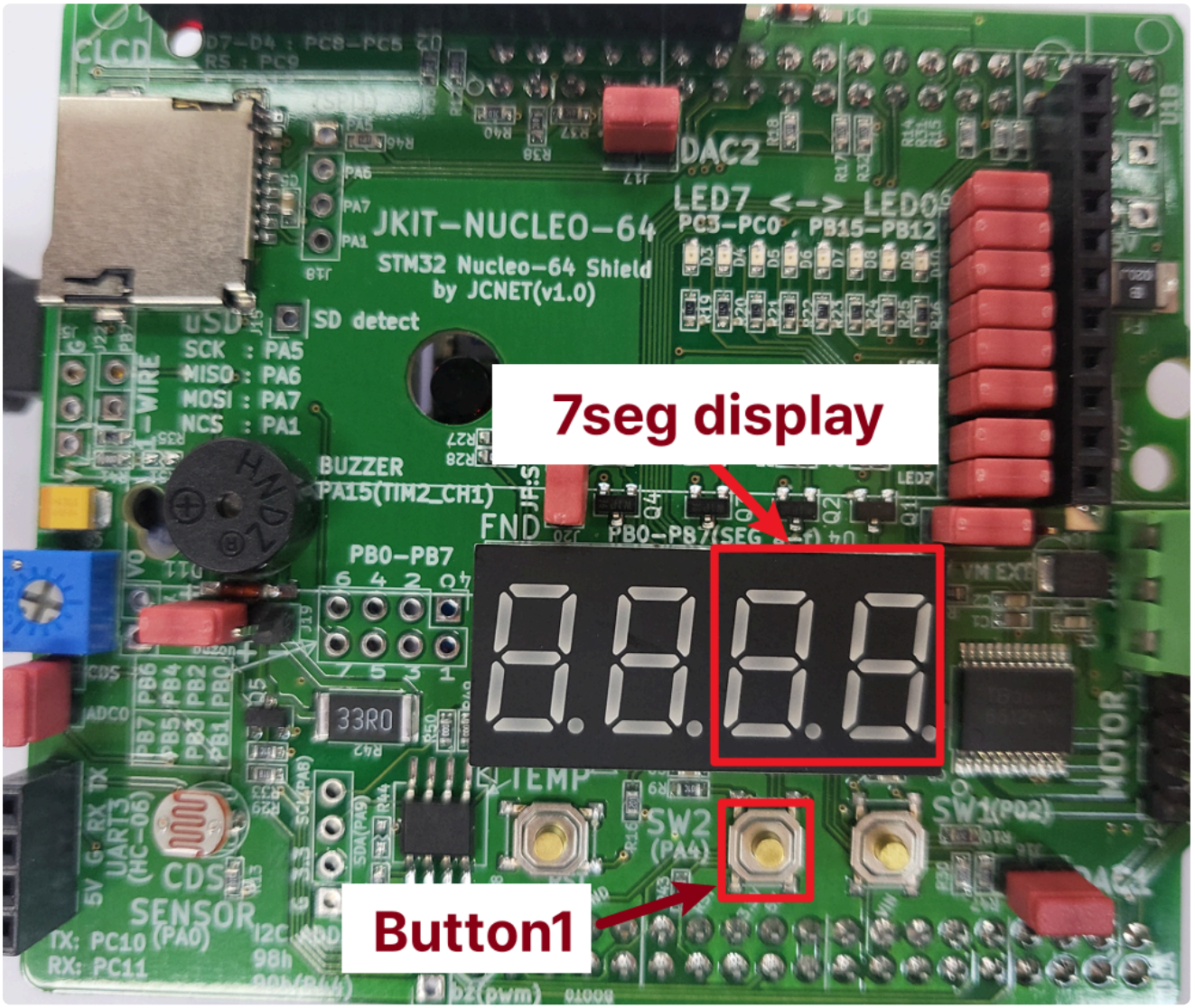
3. **setup()**
   - **System Clock & GPIO:** Initializes the HSI system clock, segment pins (`pinFND`), and digit selector pins (`selectFND`).
   - **7-segment display Configuration**: Configures pinFND and select FND
   - **Button Configuration:** Configures `BUTTON_PIN` as input with an internal pull-up resistor.
   - **Initial State:** Displays default values (`00`) on both digits prior to entering the main loop.
4. **main()**
   - **Initialization:** Declares button state variables (`currButtonState` , `prevButtonState`), initializes state tracking, and calls `setup()` .
   - **loop()**
5. **loop()**
   - **Read:** Polls `BUTTON_PIN` to capture the current input state.
   - **Process (State Update & Debounce):** Detects a falling edge (button press), increments `state` ($0 \rightarrow 19$), calculates tens (`numDisplay2`) and units (`numDisplay1`) digits, and pauses for `BUTTON_DLAY` to ignore mechanical contact chatter.
   - **Write (Multiplexing):** Alternates output between display 1 and display 2 using `SEVEN_SEG_DLAY` time-sharing to render flicker-free numbers without ghosting.

```
/*-----------------------------------------------------------------\
@ Embedded Controller by Young-Keun Kim - Handong Global University
Author           : [ Juyoung Lee ]
Created          : 09-11-2026
Modified         : 09-12-2026
Language/ver     : C in VS Code

Description      : Control 7-segments display
/-----------------------------------------------------------------*/

#include "stm32f4xx.h"
#include "ecGPIO2.h"
#include "ecRCC2.h"

#define BUTTON_PIN      PA_4
#define BUTTON_DLAY     30

#define SEVEN_SEG_DLAY  5

#define NUM_STATE       20
```

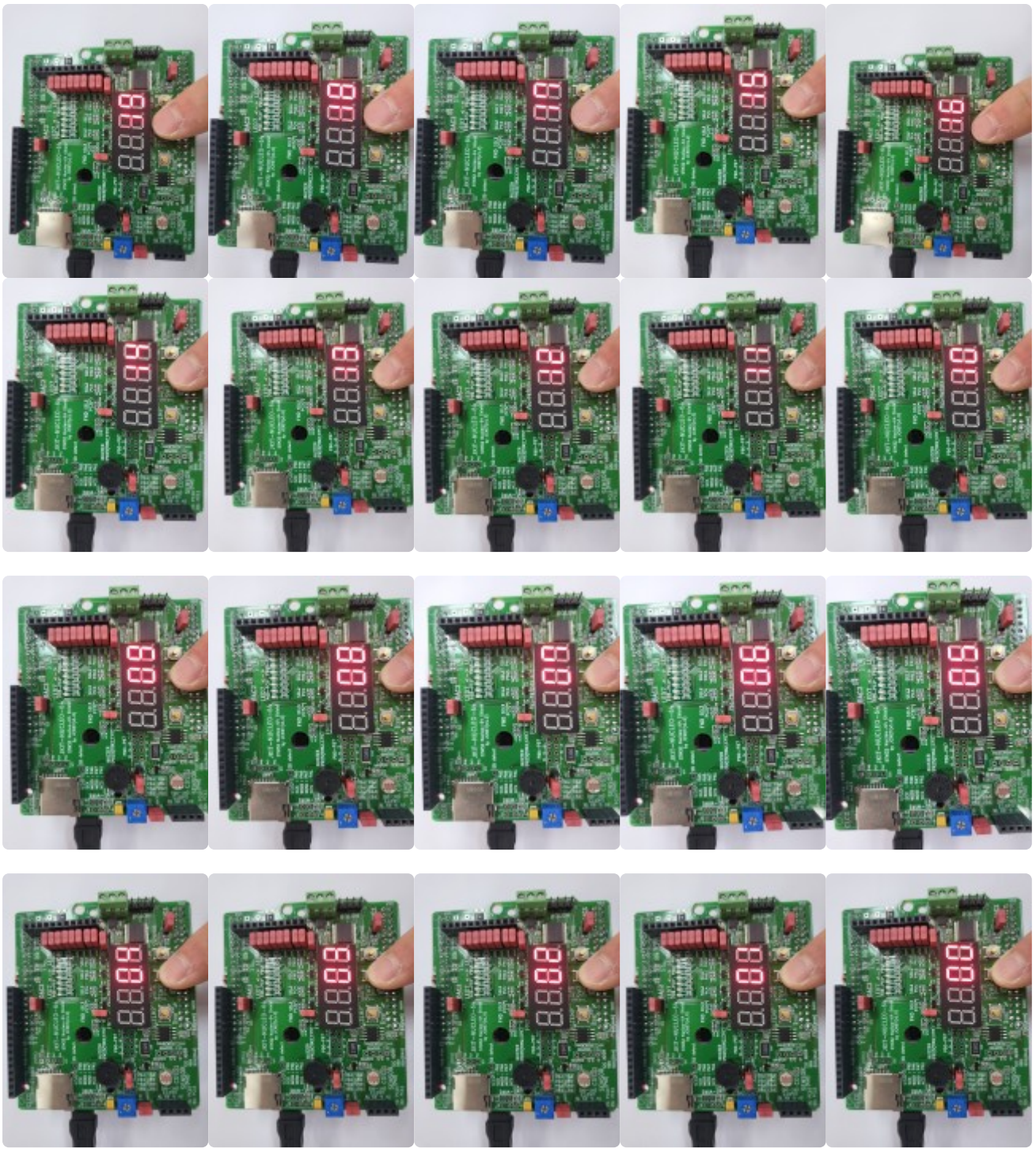
```
PinName_t pinFND[8]     = {PB_0, PB_1, PB_2, PB_3, PB_4, PB_5, PB_6, PB_7};
PinName_t selectFND[4]  = {PA_10, PA_11, PC_4, PC_3};

uint8_t numDisplay1     = 0;
uint8_t numDisplay2     = 0;
uint8_t selectFND1      = 0;
uint8_t selectFND2      = 1;

void setup(void){
    // Intialize System Clock
    RCC_HSI_init();

    // Intialize FND pins and Button
    FND_display_init(pinFND);
    FND_select_init(selectFND);

    GPIO_init(BUTTON_PIN, INPUT);
    GPIO_pupd(BUTTON_PIN, PULL_UP);

    // set state 0
    FND_select(selectFND1);
    FND_display(numDisplay1);
    FND_select(selectFND2);
    FND_display(numDisplay2);

};

int main(void) {

    uint8_t state = 0;
    uint32_t currButtonState = HIGH;
    uint32_t prevButtonState = HIGH;

    setup();

    while(1) {

        // 1) Read data
        currButtonState = GPIO_read(BUTTON_PIN);

        // 2) Process data
        if(currButtonState == LOW && prevButtonState == HIGH){

            // 2-1) Set state and display digits
            state       = (state + 1) % NUM_STATE;
            numDisplay1 = state % 10;
            numDisplay2 = state / 10;

            // 2-2) Protect from jittering
            delay_ms_HSI(BUTTON_DLAY);
```

```
        }

        // 3) Write data
        FND_select(selectFND1);
        FND_display(numDisplay1);
        delay_ms_HSI(SEVEN_SEG_DLAY);

        FND_select(selectFND2);
        FND_display(numDisplay2);
        delay_ms_HSI(SEVEN_SEG_DLAY);

        // 2-3) Update button state
        prevButtonState = currButtonState;
    }
}
```

Connection Diagram



Results



Demo Video

[EC 26-2] LAB (GPIO 7-segment)

Discussion

1. Analyze the result and explain any other necessary discussion.

> The experiment demonstrated multi-digit 7-segment display control and state-machine logic using STM32 GPIO peripherals.

- **Problem 1 (Static Multi-Digit Display):** Toggling digit selection pins `selectFND1` (PA_10) and `selectFND2` (PA_11) with a 1 ms interval rendered "50" across two digits without visible flicker, validating Persistence of Vision (POV).

- **Problem 2 (Button Counter 0–19):** Active-low button inputs from PA_4 were debounced using a 30 ms delay (`BUTTON_DLAY`), effectively preventing mechanical contact chatter from triggering false increments. State transitions smoothly rolled over from 19 back to 0 using modulo arithmetic (`(state + 1) % 20`), rendering units and tens digits via time-sharing (`SEVEN_SEG_DLAY = 5ms`).

2. What are the common cathode and common anode of 7-segment display?

> - **Common Cathode (CC):** The negative terminals (cathodes) of all segment LEDs (A–G, DP) are tied together to a common ground or active-high digit select pin. An individual segment turns **ON** when its corresponding MCU pin outputs **HIGH** (1). The JKIT board used in this lab uses a Common Cathode configuration.

> - **Common Anode (CA):** The positive terminals (anodes) of all segment LEDs are tied together to a common supply or active-low digit select pin. An individual segment turns **ON** when its corresponding MCU pin outputs **LOW** (0).

3. Does the LED of a 7-segment display (common anode) pin turn ON when 'HIGH' is given to the LED pin from the MCU?

> **No**, the LED will **not** turn ON when a **HIGH** state is applied to the segment pin on a Common Anode display.

Because the shared anode is tied to (HIGH), driving the segment cathode HIGH creates a zero voltage differential (0), keeping the diode unpowered. To illuminate the segment, the MCU must sink current by driving the segment pin **LOW** (0).

4. How can we display 4 digit-numbers on the 4 FNDS at the same time? How to remove ghosting of displaying a number?

> **Simultaneous 4-Digit Display:** Digits are refreshed in rapid succession (Digit 0 $\rightarrow$ Digit 1 $\rightarrow$ Digit 2 $\rightarrow$ Digit 3) using Time-Division Multiplexing. Keeping each digit enabled for 2–5 ms allows the full 4-digit refresh cycle to complete within 8–20 ms (0 refresh rate). Due to human visual persistence, all four digits appear illuminated at the same time.

Reference

STMicroelectronics, *STM32F411xC/E Reference Manual (RM0383)*, Rev 4, 2025. [Online]. Available: STMicroelectronics RM0383

Troubleshooting

Problem: Both 7-Segment Screens Show the Same Digits (Missing Delay)

- **Why it happens:** Without a delay (`delay_ms_HSI`) after updating each digit, the microcontroller switches between Digit 1 and Digit 2 instantly. As a result, the last number sent overwrites both digits, making both screens display the same number at the same time.
- **How to fix:** Add a delay right after displaying each digit so it has enough time to stay lit before switching to the next digit.